

1

1.1 Sizeof und Arrays

a)

Das Array wird mit vom Typ `int` Initialisiert. Eine Variable vom Typ `int` benötigt 4 Byte Speicherplatz. Somit braucht das Array insgesamt $4 \text{ Byte} \cdot 8 = 32 \text{ Byte}$ Speicherplatz. 0.5/0.5

b)

`sizeof(a)` gibt die Gesamtgröße des Arrays in Byte zurück. Siehe a) => es ergibt sich eine Gesamtgröße von 32 Byte.

Um die Anzahl der Elemente im Array zu berechnen kann man die Gesamtgröße des Arrays durch die Größe eines einzelnen Elements teilen: `sizeof(a) / sizeof(a[0])`. Dies ergibt $32 \text{ Byte} / 4 \text{ Byte} = 8$ Elemente.

```
int a [8];
int i;

for (i = 0; i < sizeof(a) / sizeof(a[0]); i++) {
    a[i] = 1;
}
```

1/1

c)

Der das Ergebnis wird 8 sein, da die Operation die Speichergröße des Datentyps des Pointers zurück gibt. 0.5/0.5

d)

Ein möglichkeit wäre die Übergabe der Arraygröße als zusätzlichen Parameter an die Funktion, da in C "Array Decay" ein

Array in einen Pointer umwandelt, wenn es an eine Funktion übergeben wird.

1/1

richtig, aber wie würde die konkrete Berechnung aussehen

1.2

a)

Zeit (ms)	Ereignis	Laufender Prozess	A	B	C	A Laufzeit	B Laufzeit	C Laufzeit	Ready-Queue	HDD-Queue (35 ms)	USB-Queue (45 ms)
Start	-	-	Rechenbereit	Rechenbereit	Rechenbereit	0	0	0	A, B, C	-	-
0	Scheduler wählt A	A	Rechnend	Rechenbereit	Rechenbereit	5			B, C	-	-
5	-	A	Rechnend	Rechenbereit	Rechenbereit	10			B, C, A	-	-
10	Scheduler wählt B	B	Rechenbereit	Rechnend	Rechenbereit		5		C, A	-	-
15	B blockiert, C wird gewählt	C	Rechenbereit	Blockiert	Rechnend			5	A	-	B
20	-	C	Rechenbereit	Blockiert	Rechnend			10	C	-	B
25	Scheduler wählt A	A	Rechnend	Blockiert	Rechenbereit	15			C	-	B
30	A blockiert, C wird gewählt	C	Blockiert	Blockiert	Rechnend			15	-	A	B
35	-	C	Blockiert	Blockiert	Rechnend			20	C	A	B
40	Scheduler wählt C	C	Blockiert	Blockiert	Rechnend			25	-	A	B
45	-	C	Blockiert	Blockiert	Rechnend			30	C	A	B
50	Scheduler wählt C	C	Blockiert	Blockiert	Rechnend			35	-	A	B
55	-	C	Blockiert	Blockiert	Rechnend			40	B, C	A	B
60	Scheduler wählt B	B	Blockiert	Rechnend	Rechenbereit		10		C	A	-
65	-	B	Rechenbereit	Rechnend	Rechenbereit		15		C, A	-	-
70	Scheduler wählt C	C	Rechenbereit	Rechenbereit	Rechnend			45	A, B	-	-
75	-	C	Rechenbereit	Rechenbereit	Rechnend			50	A, B	-	-
80	Scheduler wählt A	A	Rechnend	Rechenbereit	Done	20			B	-	-
85	A blockiert, B wird gewählt	B	Blockiert	Rechnend	Done		20		-	-	A
90	-	B	Blockiert	Rechnend	Done		25		B	-	A
95	Scheduler wählt C	B	Blockiert	Rechnend	Done		30		-	-	A
100	-	B	Blockiert	Rechnend	Done		35		-	-	A
105	Scheduler wählt B	B	Blockiert	Rechnend	Done		40		-	-	A
110	-	B	Blockiert	Rechnend	Done		45		-	-	A
115	Scheduler wählt B	B	Blockiert	Rechnend	Done		50		-	-	A
120	B ist done, Scheduler wartet	-	Blockiert	Done	Done				-	-	A
125	-	-	Blockiert	Done	Done				A	-	A
130	Scheduler wählt A	A	Rechnend	Done	Done	25			A	-	-
135	-	A	Rechnend	Done	Done	30			-	-	-
140	Scheduler wählt A	A	Rechnend	Done	Done	35			A	-	-
145	-	A	Rechnend	Done	Done	40			-	-	-
150	Scheduler wählt A	A	Rechnend	Done	Done	45			A	-	-
155	-	A	Done	Done	Done	50			-	-	-

3/3

1.3

a)

Header	Meaning	Erklärung
USER	User	Der Benutzername des Eigentümers des Prozesses
PID	Process ID	Die eindeutige Identifikationsnummer des Prozesses
PPID	Parent Process ID	Die Prozess-ID des übergeordneten Prozesses
%CPU	CPU usage	Der prozentuale Anteil der CPU-Zeit, den der Prozess seit seinem Start verwendet hat

Header	Meaning	Erklärung
%MEM	Memory usage	Der prozentuale Anteil des gesamten physischen Arbeitsspeichers, den der Prozess nutzt
TT	Terminal	Das Terminal, mit dem der Prozess verknüpft ist
STAT	Process state	Der aktuelle Zustand des Prozesses
STARTED	Start time/date	Wann der Prozess gestartet wurde
TIME	CPU time	Die gesamte CPU-Zeit, die der Prozess bisher verwendet hat
CMD	Command	Der Befehl (und seine Argumente), mit dem der Prozess gestartet wurde

1/1

b)

Der Prozess 3787 hat den Statuscode S+, welcher sich wie folgt aufschlüsseln lässt:

S: interruptible sleep (waiting for an event to complete)

+: is in the foreground process group

- Der Prozess schläft und wartet auf Ereignisse (evt. Benutzereingabe)
- Er ist interaktiv und gehört zur ordergrundgruppe des Terminals

1/1

c)

```
-- 0956 root login -- mschroetter
\-- 1009 mschroe -zsh
  \-- 1929 mschroe tmux new-session
    \-- 3828 mschroe -zsh
      \--- 3967 mschroe ps -eo user,pid,ppid
```

1/1

Aufgabe 1.4: 2/3

print_file.c

```
#include <stdlib.h>
#include <fcntl.h>
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>

#define SIZE 512

int main() {

    int fd = open("./test.txt", O_RDWR);
    if (fd == -1){
        perror("could not open file");
        return EXIT_FAILURE;
    }

    char * buffer = (char *) calloc(SIZE, sizeof(char));

    if (!buffer) {
        perror("could not allocate memory");
        close(fd);
        return EXIT_FAILURE; ✓
    }

    ssize_t r = read(fd, buffer, SIZE-1);

    if (r == -1) {
        perror("could not read file");
        free(buffer);
        close(fd);
        return EXIT_FAILURE;
    }

    printf("%.5s", SIZE-1, buffer);
    free(buffer);
    close(fd);
    return EXIT_SUCCESS;
}
```


workdir.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

char *mywdfunc(unsigned int bufsize) {
    if (bufsize < 100) {
        return NULL;
    }

    char *buf = (char *)calloc(bufsize, sizeof(char));
    if (buf) {
        char *cwd = getcwd(buf, bufsize-1);
        if (!cwd) {
            free(buf);
            return NULL;
        }
    }

    return buf;
}

int main() {
    char *ret = mywdfunc(512);

    printf("%s", ret);
    free(ret);
    return EXIT_SUCCESS;
}
```

Rückgabewert von mywdfunc() ebenfalls auf NULL prüfen, um Fehlerfall zu erkennen. -1

Aufgabe 1.5: 6/7

dl_proc_list.h 1/1

```
typedef struct dl_proc_node {  
    int pid;  
    int ppid;  
    struct dl_proc_node* prev; ✓  
    struct dl_proc_node* next;  
} dl_proc_node;
```

```
typedef struct dl_proc_list {  
    dl_proc_node* head; ✓  
} dl_proc_list;
```

```
dl_proc_list* dl_proc_list_create(void);
```

```
int dl_proc_list_insert(dl_proc_list *list, int pid);
```

```
int dl_proc_list_get(dl_proc_list *list, int position, int* ✓  
pid, int* ppid);
```

```
int dl_proc_list_remove(dl_proc_list *list, int pid);
```

```
void dl_proc_list_free(dl_proc_list *list);
```

Makefile vorhanden. 0,5/0,5

dl_proc_list.c 3/3,5

```
#include "dl_proc_list.h"
#include <stdlib.h>

// Creates a new process list with the initial process (PID = 1, PPID = 0)
dl_proc_list* dl_proc_list_create() {
    // Create list
    dl_proc_list *list;
    list = (dl_proc_list *) malloc(sizeof(dl_proc_list));

    if (!list) {
        return NULL; ✓
    }

    // Create first process node
    dl_proc_node *first_node;
    first_node = (dl_proc_node *)
malloc(sizeof(dl_proc_node));

    if (!first_node) {
        free(list); ✓
        return NULL;
    }

    // Initialize first process node (PID = 1, PPID = 0)
    first_node->pid = 1;
    first_node->ppid = 0; ✓

    first_node->prev = NULL;
    first_node->next = NULL; ✓

    // Set head of list to first node
    list->head = first_node; ✓

    return list;
}

// Inserts a new process with given PID into process list
// Returns 1 on success, -1 on memory allocation failure,
// -2 on invalid args, -3 if PID already exists
int dl_proc_list_insert(dl_proc_list *list, int pid){
    // Invalid list
    if (!list) {
        return -2; ✓
    }

    // Invalid PID
    if (pid <= 1) {
        return -2; ✓
    }
}
```

```

}

// Find position to insert new process node
dl_proc_node *pre_new = list->head;

while (pre_new && pre_new->next && pre_new->next->pid <
pid) {
    pre_new = pre_new->next;
}

// PID already exists
if ((pre_new && pre_new->pid == pid) ||
    (pre_new && pre_new->next && pre_new->next->pid ==
pid)) {
    return -3; ✓
}

// pre_new is either the end of the list or the last node
with PID < new PID

// Invalid position
if (!pre_new) {
    return -3;
}

// Create new process node
dl_proc_node *new_node = (dl_proc_node *)
malloc(sizeof(dl_proc_node));
if (!new_node) {
    return -1; ✓
}

new_node->pid = pid; ✓

if (pre_new->next) {
    new_node->ppid = pre_new->next->pid; // smaller
} else {
    new_node->ppid = pre_new->pid; // larger inserted
}

new_node->prev = pre_new;
new_node->next = pre_new->next; ✓

if (pre_new->next) {
    pre_new->next->prev = new_node;
}
pre_new->next = new_node; ✓

```

Es können weitere Elemente in der Liste existieren, deren PID größer ist als von pre_new->next.

inserted before larger
after smaller
Liefert nicht notwendigerweise die bisher höchste Prozess-ID. -0.5 ✓

```

    return 0;
}

int dl_proc_list_get(dl_proc_list *list, int position, int*
pid, int* ppid){
    // Invalid list
    if (!list) {
        return -1; ✓
    }

    dl_proc_node *current = list->head;
    int index = 0;

    while (current && index < position) {
        current = current->next;
        index++; ✓
    }

    // Position out of bounds
    if (!current) {
        return -2; ✓
    }

    *pid = current->pid;
    *ppid = current->ppid; ✓

    return 0;
}

```

```

int dl_proc_list_remove(dl_proc_list *list, int pid){
    // Invalid list
    if (!list) {
        return -2; ✓
    }

    // Invalid PID, cannot remove PID 1
    if (pid <= 1) {
        return -2; ✓
    }

    dl_proc_node *current = list->head;

    while (current && current->pid != pid) {
        current = current->next; ✓
    }

    // PID not found
    if (!current) {
        return -3; ✓
    }
}

```

```

}

int parent_pid = current->ppid;

// Remove the node from the list
if (current->prev) {
    current->prev->next = current->next;
} else {
    // Removing head node ✓
    list->head = current->next;
}

if (current->next) {
    current->next->prev = current->prev; ✓
}

free(current); ✓
dl_proc_node *node = list->head;
while (node) {
    if (node->ppid == pid) {
        node->ppid = parent_pid; ✓
    }
    node = node->next;
}

return 0;
}

void dl_proc_list_free(dl_proc_list *list){
    if (!list) {
        return; ✓
    }

    dl_proc_node *current = list->head;
    dl_proc_node *next_node;

    while (current) {
        next_node = current->next;
        free(current); ✓
        current = next_node;
    }

    free(list); ✓
}

```

proctable.c 1,5/2

```
#include <stdio.h>
#include <stdlib.h>
#include "dl_proc_list.h"

int main(int argc, char *argv[]) {
    dl_proc_list *plist = dl_proc_list_create();
    if (!plist) {
        fprintf(stderr, "Fehler: Konnte Prozesstabelle nicht
erstellen.\n");
        return EXIT_FAILURE; ✓
    }

    for (int i = 1; i < argc; i++) {
        char *endptr;
        long val = strtol(argv[i], &endptr, 10);

        if (*endptr != '\0') {
            fprintf(stderr, "Warnung: Ungueltiges Argument
\"%s\" ignoriert.\n", argv[i]); ✓
            continue;
        }

        if (val == 0) {
            fprintf(stderr, "Warnung: PID = 0 wird ignoriert.
\n");
            continue; ✓
        }

        if (val > 0) {
            if (dl_proc_list_insert(plist, (int)val) < 0) {
                fprintf(stderr, "Fehler beim Einfuegen: PID
%d existiert bereits.\n", val); ✓
            }
        } else {
            int pid_to_remove = (int)(-val);
            if (pid_to_remove == 1) {
                fprintf(stderr, "Fehler: Prozess mit ID = 1
kann nicht geloescht werden.\n"); ✓
            } else if (dl_proc_list_remove(plist,
pid_to_remove) < 0) {
                fprintf(stderr, "Fehler beim Loeschen: Prozess
mit ID = %d existiert nicht.\n", pid_to_remove); ✓
            }
        }
    }

    int pos = 0, pid, ppid;
    while (dl_proc_list_get(plist, pos, &pid, &ppid) == 0) {
        printf("PID = %d, Parent PID = %d\n", pid, ppid); ✓
        pos++;
    }
}
```

strtol() liefert einen Wert vom Typ long zurück. Vor dem Casting nach int darauf achten, dass der Wert nicht unter INT_MIN bzw. über INT_MAX liegt. -0,5

```
    }  
    dl_proc_list_free(plist);  
    return EXIT_SUCCESS; ✓  
}
```


Übungsblatt 2

Donnerstag, 13. November 2025 22:28

Yunis Drescher, 827130 Joshua Neunert, 822456 Konrad Gose, 828440

10/11 Aufgabe 2.2: Scheduling

- 5/5** a) In der Übung wurde die FiFo-Scheduling-Strategie in den **Sched-Sim** Simulator implementiert. Erweitern Sie **schedule.c** um die Implementierung der Round-Robin-, Shortest-Job-First- und Fixed-Priority-Scheduling-Strategie. Erweitern Sie falls nötig die Implementierung der Run-Queue in **queue.c** um die benötigten Funktionen. (5 Punkte)
- 2/3** b) Erstellen Sie für die Algorithmen Round-Robin, Shortest-Job-First und Fixed-Priority je ein Gantt-Diagramm, wobei die Kosten für Scheduler- und Dispatcher vernachlässigt werden sollen. Übernehmen Sie die Laufzeit und Ankunftszeiten der Prozess aus der **main.c**.
Hinweis: Verwenden Sie für ihre Zeichnung ein Quantum von 100ms. (3 Punkte)
- 3/3** c) Bewerten Sie die **Effizienz** der drei Scheduling-Strategien in Bezug auf die mittlere Wartezeit, die mittlere Verweildauer und die mittlere Antwortzeit. Beziehen Sie sich dabei auf die in Aufgabe b erstellten Ablaufdiagramme. (3 Punkte)

FiFo													
Prozess	0	100	200	300	400	500	600	700	800	900	1000	1100	1200
1					a								
2	a												
3			a										
4	a												
Wartezeit		600											
Verweilzeit			700										
Antwortzeit				600									
1		600											
2		150											
3		500											
4		0											
Durchschnitt		312,5											
Round Robin													
Prozess	0	100	200	300	400	500	600	700	800	900	1000	1100	1200
1					a								
2	a												
3			a										
4	a												
Wartezeit		200											
Verweilzeit			300										
Antwortzeit				200									
1		200											
2		550											
3		500											
4		100											
Durchschnitt		337,5											
SJF													
Prozess	0	100	200	300	400	500	600	700	800	900	1000	1100	1200
1					a								
2	a												
3			a										
4	a												
Wartezeit		0											
Verweilzeit			100										
Antwortzeit				100									
1		0											
2		550											
3		100											
4		0											
Durchschnitt		162,5											
Priority													
Prozess	0	100	200	300	400	500	600	700	800	900	1000	1100	1200
1					a								
2	a												
3			a										
4	a												
Wartezeit		0											
Verweilzeit			100										
Antwortzeit				100									
1		0											
2		150											
3		500											
4		900											
Durchschnitt		387,5											
1. Platz		SJF = 162,5											
2. Platz		FiFo = 312,5											
3. Platz		RR = 337,5											
4. Platz		Prio = 387,5											
Wartezeit													
Verweilzeit													
Antwortzeit													
1. Platz		SJF = 162,5											
2. Platz		FiFo = 312,5											
3. Platz		RR = 337,5											
4. Platz		Prio = 387,5											
Insgesamt													
1. Platz		SJF											
2. Platz		RR											
3. Platz		FiFo											
4. Platz		Prio											

Aufgabe 2.3: Windows Scheduler

Gegeben sei ein **Einprozessor-System** mit Windows 2000. Weiterhin existieren 3 Benutzerprozesse mit der **regulären BasePriority**, von denen zwei rechenbereit sind. Bearbeiten Sie die drei nachfolgenden Szenarien mit den dazugehörigen Teilaufgaben.

- a) Im ersten Szenario ist (Prozess 2) auf das Netzwerk **wartend blockiert**. Prozess 2 erhält zum Zeitpunkt $t=0$ die CPU, wenn das Netzwerk bereit ist.
- Ermitteln Sie die Größe der Zeitscheibe von Prozess 2.
 - Ermitteln Sie die Anzahl an Sekunden, bis Prozess 2 seine **Basispriorität** wieder erreicht hat.
 - Zeichnen Sie ein modifiziertes Gantt-Diagramm, aus dem die aktuellen Prioritäten der Prozesse ablesbar sind. Das Diagramm muss für jeden Prozess **mindestens 4 Zeitscheiben** umfassen.
- b) Im zweiten Szenario ist Prozess 2 ein **GUI-Thread**. Bearbeiten Sie die Teilaufgaben a)(i)-a)(iii) **erneut** für diesen Fall.

Hinweis: Benutzen Sie die Boost- und Zeitscheibenwerte aus der Vorlesung!

(1,5 + 1,5 Punkte)

a)

i)

Wir wissen aus der Vorlesung, das für Windows2000 Zeitscheiben 20 ms groß sind. Da Prozess 2 keine Interaktive Sitzung ist, wird die Zeitscheibe auch nicht für den boost verdreifacht.

Also ist die Zeitscheibe für Prozess 2 20 ms groß

ii)

Da der Thread auf das Netzwerk wartet, ist der I/O - Boost System = 2

Also ist die Priority danach

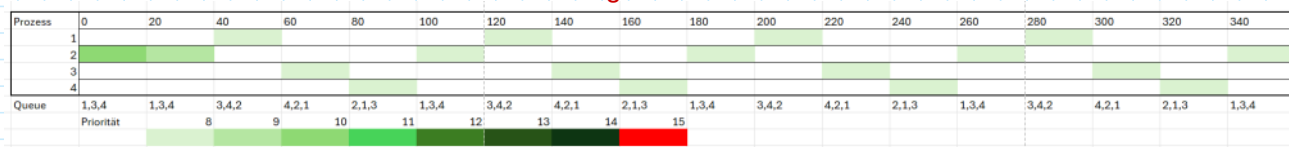
$$\min\{8+2, 15\} = 10$$

Da ein Prozess jedes Quantum einen Priority boost verliert, benötigt es zwei Quanta um wieder zur Basispriorität zu gelangen

Also 40 ms = 0,04 s

iii)

es gibt nur 3 Prozesse



b)

i)

Da der Prozess 2 nun eine Interaktive Sitzung ist, ist die erste Zeitscheibe verdreifacht nach dem Boost.

Also hat der Prozess einmalig eine Zeitscheibe von 60ms und danach wieder eine 20ms Zeitscheibe

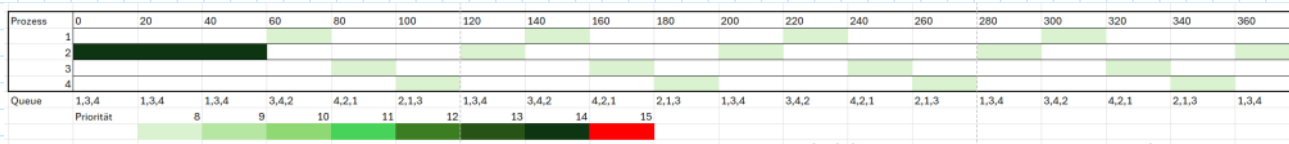
ii)

Aus der Vorlesung haben wir, das Interaktive Sitzungen einen Boost von 6 bekommen, also

$$\min\{8+6, 15\} = 14$$

aber der Boost hält nur für eine Zeitscheibe. Da die Zeitscheibe dafür aber verdreifacht ist, braucht der Prozess ein dreifaches Quanta oder 60 ms. Das sind 0,06 Sekunden, bis der Prozess wieder seine Basis Priorität erreicht.

iii)



auch hier nur 3 Prozesse gegeben

1/1.5

zu b)

Round Robin: P3 wird bei Ankunft bei 200 ms vor P2 eingeordnet und startet ab 300 ms. -0,5 Analog wird bei der Ankunft von P1 vorgegangen, wodurch P1 bereits ab 500 ms startet.

Die richtige Reihenfolge lautet also: P4, P2, P4, P3, P2, P1, ...

SJF: P3 wird nicht durch die Ankunft von P1 bei 400 ms unterbrochen. -0,5 SJF arbeitet nicht-verdrängend.

a) 5/5

queue.c

```
#include <queue.h>
#include <stdlib.h>
#include <stddef.h>
#include <process.h>
#include <schedule.h>
#include <internal/misc.h>

int queue_enqueue_rr(process_queue_t *queue, process_t
*process);
int queue_enqueue_sjf(process_queue_t *queue, process_t
*process);
int queue_enqueue_prio(process_queue_t *queue, process_t
*process);

void queue_set_functions(void)
{
    // setting queue functions
    global_state.queue_functions.queue_init = queue_init;
    global_state.queue_functions.queue_enqueue =
queue_enqueue;
    global_state.queue_functions.queue_destroy =
queue_destroy;
    global_state.queue_functions.queue_length = queue_length;
    global_state.queue_functions.queue_get_process_at =
queue_get_process_at;

    // If you want that new processes are added in a different
way for one scheduler
    if (schedule == schedule_sjf)
    {
        global_state.queue_functions.queue_enqueue =
queue_enqueue_sjf;
    }
    else if (schedule == schedule_prio)
    {
        global_state.queue_functions.queue_enqueue =
queue_enqueue_prio;
    }
    else if (schedule == schedule_rr)
    {
        global_state.queue_functions.queue_enqueue =
queue_enqueue_rr;
    }

    return;
}

int queue_init(process_queue_t **queue)
{
    *queue = (process_queue_t
```

```

*)malloc(sizeof(process_queue_t));
    if (*queue == NULL)
    {
        return -1;
    }
    (*queue)->head = NULL;
    (*queue)->tail = NULL;
    (*queue)->length = 0;
    return 0;
}

// Default
int queue_enqueue(process_queue_t *queue, process_t *process)
{
    process_node_t *new_node = (process_node_t
*)malloc(sizeof(process_node_t));
    if (new_node == NULL)
    {
        return -1;
    }
    new_node->process = process;
    new_node->prev = queue->tail;
    new_node->next = NULL;

    if (queue->tail)
    {
        queue->tail->next = new_node;
    }
    else
    {
        queue->head = new_node;
    }
    queue->tail = new_node;
    queue->length++;
    return 0;
}

// Round Robin
int queue_enqueue_rr(process_queue_t *queue, process_t
*process)
{
    // args validation
    if (!queue || !process)
    {
        return -1; ✓
    }

    process_node_t *new_node = (process_node_t
*)malloc(sizeof(process_node_t));
    if (new_node == NULL) ✓

```

```

{
    return -1;
}

new_node->process = process;
new_node->prev = queue->tail;
new_node->next = NULL;

if (queue->tail)
{
    // Add to the end
    queue->tail->next = new_node; ✓
}
else
{
    // Default set as head
    queue->head = new_node; ✓
}

queue->tail = new_node;
queue->length++; ✓
return 0;
}

// Shortest-Job-First
int queue_enqueue_sjf(process_queue_t *queue, process_t
*process)
{
    // args validation
    if (!queue || !process)
    {
        return -1; ✓
    }

    process_node_t *new_node = (process_node_t
*)malloc(sizeof(process_node_t));
    if (new_node == NULL)
    {
        return -1; ✓
    }

    new_node->process = process;
    new_node->prev = NULL;
    new_node->next = NULL;

    if (queue->head == NULL)
    {
        // Insert as Head
        queue->head = new_node; ✓
        queue->tail = new_node;
    }
}

```

```

    }
    else
    {
        process_node_t *current = queue->head;
        while (current != NULL && current->process->burst_time
<= process->burst_time)
        {
            current = current->next;
        }
        if (current == queue->head)
        {
            new_node->next = queue->head;
            queue->head->prev = new_node; ✓
            queue->head = new_node;
        }
        else if (current == NULL)
        {
            new_node->prev = queue->tail;
            queue->tail->next = new_node; ✓
            queue->tail = new_node;
        }
        else
        {
            new_node->next = current;
            new_node->prev = current->prev; ✓
            current->prev->next = new_node; ✓
            current->prev = new_node;
        }
    }
    queue->length++; ✓
    return 0;
}

```

// Fixed-Priority

```

int queue_enqueue_prio(process_queue_t *queue, process_t
*process)
{
    process_node_t *new_node = (process_node_t
*)malloc(sizeof(process_node_t));
    if (new_node == NULL) ✓
    {
        return -1;
    }

    new_node->process = process;
    new_node->prev = NULL;
    new_node->next = NULL;

    if (queue->head == NULL)
    {

```

```

        queue->head = new_node; ✓
        queue->tail = new_node; ✓
    }
    else
    {
        process_node_t *current = queue->head;
        while (current != NULL && current->process->priority
<= process->priority)
        {
            current = current->next;
        }
        if (current == queue->head)
        {
            new_node->next = queue->head; ✓
            queue->head->prev = new_node; ✓
            queue->head = new_node;
        }
        else if (current == NULL)
        {
            new_node->prev = queue->tail; ✓
            queue->tail->next = new_node;
            queue->tail = new_node;
        }
        else
        {
            new_node->next = current;
            new_node->prev = current->prev; ✓
            current->prev->next = new_node; ✓
            current->prev = new_node;
        }
    }
    queue->length++; ✓
    return 0;
}

```

```

int queue_dequeue(process_queue_t *queue, process_t **process)
{
    if (queue->head == NULL)
    {
        return -1;
    }
    process_node_t *node_to_remove = queue->head;
    *process = node_to_remove->process;
    queue->head = node_to_remove->next;

    if (queue->head)
    {
        queue->head->prev = NULL;
    }
    else

```



```

    {
        queue->tail = NULL;
    }
    free(node_to_remove);
    queue->length--;
    return 0;
}

int queue_length(process_queue_t *queue)
{
    return queue->length;
}

int queue_get_process_at(process_queue_t *queue, process_t
**process, int index)
{
    if (index < 0 || index >= (int)queue->length)
    {
        return -1;
    }
    process_node_t *current = queue->head;
    for (int i = 0; i < index; i++)
    {
        current = current->next;
    }
    *process = current->process;
    return 0;
}

int queue_destroy(process_queue_t *queue)
{
    process_node_t *current = queue->head;
    while (current != NULL)
    {
        process_node_t *next = current->next;
        free(current);
        current = next;
    }
    free(queue);
    return 0;
}

```

schedule.c

```
#include <schedule.h>
#include <process.h>
#include <queue.h>
#include <internal/misc.h>

int schedule_fifo(process_t *current_process, process_t
**next_process) {
    if(current_process && current_process->state == READY) {
        *next_process = current_process;
        return 0;
    }

    if (queue_length(current_core->run_queue) < 1) {
        return -1;
    }

    return queue_dequeue(current_core->run_queue, next_process);
}

// Round Robin
int schedule_rr(process_t *current_process, process_t
**next_process) {
    int ret;
    process_t *next = NULL;

    // first element in the run queue
    ret = queue_dequeue(current_core->run_queue, &next);

    if (ret == 0) {

        // append the current process back to the run queue if
        it's still READY
        if (current_process && current_process->state == READY) {
            int enq_ret = queue_enqueue(current_core->run_queue,
current_process);
            if (enq_ret != 0) { ✓
                return enq_ret;
            }
        }

        // assign the next process to run
        *next_process = next; ✓
        return 0;
    } else {
        // empty queue

        // let the current process continue if it's still READY
        if (current_process && current_process->state == READY) {
            *next_process = current_process; ✓
        }
    }
}
```

```

        return 0;
    }

    // queue is empty
    return -1; ✓
}

// Shortest Job First
int schedule_sjf(process_t *current_process, process_t
**next_process) {
    int len = 0;
    // If a process is running continue
    if(current_process && current_process->state == READY) {
        *next_process = current_process; ✓
        return 0;
    }
    // If no process is running find next
    if ((len = queue_length(current_core->run_queue)) < 1) {
        return -1; ✓
    }

    process_t *shortest_job = NULL;
    process_t *temp_process = NULL;
    int ret = 0;

    // find shortest job in the run queue
    for (int i = 0; i < len; i++) {
        ret = queue_dequeue(current_core->run_queue,
&temp_process);
        if (ret != 0) { ✓
            return ret;
        }

        // Check if process is new shortest job
        if (shortest_job == NULL || temp_process->burst_time <
shortest_job->burst_time) {
            if (shortest_job != NULL) {
                ret = queue_enqueue(current_core->run_queue,
shortest_job);
                if (ret != 0) { ✓
                    return ret;
                }
            }
            shortest_job = temp_process;
        } else {
            ret = queue_enqueue(current_core->run_queue,
temp_process);
            if (ret != 0) { ✓
                return ret;
            }
        }
    }
}

```

```

    }
}

*next_process = shortest_job; ✓
return 0;
}

// Fixed Priority Scheduling
int schedule_prio(process_t *current_process, process_t
**next_process) {
    int len = queue_length(current_core->run_queue);
    int ret = 0;
    process_t *highest_in_queue = NULL;
    process_t *temp_process = NULL;

    // find highest priority process in the run queue
    if (len > 0) {
        for (int i = 0; i < len; i++) {
            ret = queue_dequeue(current_core->run_queue,
&temp_process);
            if (ret != 0) return ret; ✓

            if (highest_in_queue == NULL || temp_process->priority >
highest_in_queue->priority) {
                if (highest_in_queue != NULL) {
                    ret = queue_enqueue(current_core->run_queue,
highest_in_queue);
                    if (ret != 0) return ret; ✓
                }
                highest_in_queue = temp_process;
            } else {
                ret = queue_enqueue(current_core->run_queue,
temp_process);
                if (ret != 0) return ret; ✓
            }
        }
    }

    // continue with current process or switch to highest
priority from queue
    if (current_process && current_process->state == READY) {

        if (highest_in_queue == NULL) {
            // empty queue, continue current process
            *next_process = current_process; ✓
            return 0;
        }
    }
}

```

```

    // Compare priorities
    if (current_process->priority >= highest_in_queue-
>priority) {
        *next_process = current_process;
        ret = queue_enqueue(current_core->run_queue,
highest_in_queue);
        return ret;
    } else {
        // Switch to higher priority process
        *next_process = highest_in_queue;
        ret = queue_enqueue(current_core->run_queue,
current_process);
        return ret;
    }

} else {
    // no current process, run highest priority from queue
    if (highest_in_queue == NULL) {
        // empty queue
        return -1;
    }
    // Start the highest priority process from the queue.
    *next_process = highest_in_queue;
    return 0;
}
}

```

Übungsblatt 3

Mittwoch, 26. November 2025 21:04

2,5/4 Aufgabe 3.1: Echtzeit-Scheduling und Prioritäten

0,5/2 a) Gegeben sei die folgende Menge an Rechenaufträgen mit ihren Planungsdaten:

Prozess	Ankunftszeit	Rechenzeit	Deadline
P_1	0	4	8
P_2	2	3	10
P_3	6	4	14
P_4	6	2	10
P_5	8	5	15

Prüfen Sie, ob ein gültiger Ablaufplan für die gegebenen Aufträge existiert, wenn die Prozesse auf einem Single-Core-System nicht unterbrechend ausgeführt werden!

Hinweis: Deadline meint hier die absolute Deadline, also den entsprechenden Zeitpunkt!

Wir füllen den Plan nur Exemplarisch bei der Problematischen Stelle aus:

Prozess	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
P1	a								d							
P2			a								d					
P3							a								d	
P4							a				d					
P5									a							d

Start von P4, P3 und P5 jeweils um 1 Zeiteinheit verschoben. -0,5
P4 startet erst bei 7 statt 6, da P1 + P2 insgesamt für 7 Einheiten rechnen.

Wie man sieht, ist es nicht möglich, dass die Letzten drei Prozesse zu Problemen führen und nicht Abgeschlossen werden können. Selbst wenn man nur die Prozesse P3-5 betrachtet, überschneiden sie sich so, dass P5 nicht abgeschlossen werden kann. Wenn man Den Prozessor auf P5 warten lässt statt P4 zu starten haben wir das Problem, dass P4 nicht abgeschlossen wird.

Einfacher noch kann man sehen, dass die Gesamtrechenzeit länger als die Späteste Deadline ist.

$$4+3+4+2+5 = 18 > 15$$

Damit lassen sich die Prozesse nicht rechtzeitig ausführen.



zu a) Scheduling-Verfahren explizit nennen in Begründung. -1

z. B. Wenn Earliest-Deadline-First nicht funktioniert, dann existiert kein gültiger Ablaufplan.

- 2/2** b) Windows/NT benutzt 16 feste Prioritäten (Prio-Klassen 16 - 31). Geben Sie für folgende Prozesse geeignete Prioritäten an, so daß alle Fristen eingehalten werden! Wie sieht der zugehörige Ablaufplan aus?

Prozess	r_i	t_i	d_i
P_1	0	2	10
P_2	1	4	10
P_3	2	3	15
P_4	8	2	10
P_5	9	4	14

Mit r_i : ready time; t_i : Schätzung für maximale Ausführungszeit; d_i : Deadline.

(2 + 2 Punkte)

Wir erhalten beim Einfügen so das Prozesse P4 und P5 passen, da sie die Engsten Zeitpläne haben und dem Hinzufügen der anderen Prozesse so das alles Passt folgenden Ablaufplan:

Prozess	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
P1	a										d					
P2		a									d					
P3			a													
P4									a		d					
P5										a					d	

a= Ankunft
d= Deadline

Daraus folgt für die Prioritäten:

$P_1 < P_2$

$P_3 < P_1, P_2$

$P_3 < P_4, P_5$

$P_5 < P_4$

$\Rightarrow P_3 < P_1 < P_2, P_3 < P_5 < P_4$

Wir setzen um zu verdeutlichen dass P4 und P5 dringend sind:

$P_3 < P_1 < P_2 < P_5 < P_4$

Obwohl P4 und P5 nicht eine höhere Priority als P1 und P2 haben müssen, da sie sich nicht überschneiden

$P_1 = 23$

$P_2 = 24$

$P_3 = 22$

$P_4 = 26$

$P_5 = 25$

Wir orientieren uns dabei an der Base Priority Real-Time und haben unterschiedliche Priority Classes

Aufgabe 3.2: Linux CFS-Scheduling

Der Linux Kernel unterstützt seit der Kernel-Version 2.6.24 **Gruppen-basiertes Scheduling**. Beantworten Sie für ein **Einkern** Linux-System mit den folgenden **Gruppen** (/ und tg1) und **Prozessen** (P1, P2 und P3) die folgenden Fragen:

Der Prozess **P3** hat außerdem einen **nice-Wert** von -2, alle anderen Variablen haben ihren **Standardwert**.

- a) Geben Sie die **CFS-Runqueues** (Red-Black-Trees) an, die der Linux-Kernel für die oben genannten Prozesse und Gruppen verwaltet, indem Sie die zugeordneten **Entitäten** angeben.

a)

Der Linux CFS Scheduler legt für jedes Scheduling Entity (SE) eine Runqueue Struktur an.

SE's sind:

Prozesse wie P1, P2, P3

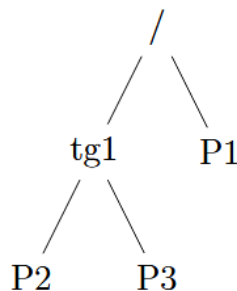
Gruppen wie die Root-Gruppe "/" und die Untergruppe tg1

Also erstellen wir Runqueues für alle SE's.

Als Entitäten erhalten die Queues entweder alle SE's die von der Gruppe abstammen oder "Ø" wenn es sich um einen Prozess handelt.

Runqueue	Entitäten
/	Tg1, P1
tg1	P2, P3
P1	Ø
P2	Ø
P3	Ø

1/1



- b) Diskutieren Sie, zu welchem Zeitpunkt ein Betriebssystem die **vruntime** einer Scheduling-Entität aktualisieren sollte. Überprüfen Sie Ihre Hypothese für den CFS-Scheduler.
- c) Angenommen der Prozess **P3** läuft als erstes auf dem System für **10ms**. Bestimmen Sie die **vruntime** aller betroffenen Scheduling-Entitäten.
- d) Bestimmen Sie die CPU-Zeit der Prozesse P1, P2 und P3 nach einer Sekunde in einem **idealen System**.

(1+1+1+2 Punkte)

b)

Sinnvolle Zeitpunkte können sein:

1. Periodisch: Durch System-Tick sicherstellen, dass lang laufende Prozesse ihre Zeit angerechnet bekommen und ggf. verdrängt werden können.
2. Zustandswechseln: Sobald ein Prozess die CPU freiwillig abgibt oder geweckt wird, damit seine Platzierung im Red-Black-Tree korrekt ist.

Der CFS aktualisiert die vruntime über den function call `update_curr()`, welcher an folgenden Stellen steht:

- `scheduler_tick()` : Periodisch bei jedem Timer-Tick (Interval Abhängig von der HZ - Konfiguration) **richtig** **1/1**
- `dequeue_entity()` / `enqueue_entity()` : Wenn ein Task schlafen geht oder aufwacht
- `put_prev_task()` : Wenn ein Task verdrängt wird oder `yield()` aufruft.

c)

$$vruntime = Laufzeit \times \frac{1024}{weight(nice)}$$

$$weight(-2) = 1586$$

$$vruntime = 10ms \times \frac{1024}{1586} \approx 6,46 ms$$

Also ist die vruntime von P3 nach 10ms 6,46ms **richtig**

Wenn bei einem Prozess die vruntime aktualisiert wird, wird sie es bei alle ihren Vorgängern auch. Das heißt das wenn bei P3 die vruntime bei 6,46ms liegt und da es der erste Prozess ist alle anderen vruntime bei 0ms sind, ist die vruntime von tg1 und "/" auch 6,46ms, also

bei tg1 ist sie allerdings 10ms (standard nice wert)

Runqueue	vruntime
/	6,46 ms
tg1	6,46 ms
P1	0 ms
P2	0 ms
P3	6,46 ms

0.5/1

d)

Time Slice basierend auf den Gewichten in der Hierarchie.

- Gesamtgewicht Root: $1024(P1) + 1024(tg1) = 2048$
- P1: 50%
- tg1: 50%

Zeit für P1: $1000ms \times 0,5 = 500ms$

Zeit für tg1: $1000ms \times 0,5 = 500ms$

die Zeit von tg1 auf SubTask aufteilen.

- Gewicht P2 (Nice 0): **1024**
- Gewicht P3 (Nice -2): **1586**
- Gesamtgewicht in tg1: $1024 + 1586 = 2610$

Zeit für P2:

$$\text{Zeit}_{P2} = 500ms \times \frac{1024}{2610} \approx 500ms \times 0,3923 = 196,17ms$$

Zeit für P3:

$$\text{Zeit}_{P3} = 500ms \times \frac{1586}{2610} \approx 500ms \times 0,6077 = 303,83ms$$

Prozess	CPU-Zeit
P1	500,00 ms
P2	196,17 ms
P3	303,83 ms

2/2

Aufgabe 3.3: 3,5/7

a) 3,5/4

```
#include <stdio.h>
#include "animal.h"
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>
#include <pthread.h>

//static char *words[29];
static double *probability;

void* thread_activities(void *args){
    void **data = (void**)args; // pointer auf void to convert
    stuff
    int j = *(int*) data[0]; // convert void Pointer zu int and
    derefenziere ihn (get the index i)
    char *word = (char*)data[1]; // convert void pointer zu char
    *(probability + j) = animal_probability(word);
    free(data[0]);
    free(args); ✓
    return NULL;
}

int main() {
    char *words[] = {"tiger", "river", "otter", "coffee",
                    "walrus", "router", "yak", "algorithm",
                    "dog", "cat", "math", "bike",
                    "elephant", "table", "guitar", "piano",
                    "lion", "desk", "window", "book",
                    "fish", "banana", "computer", "phone",
                    "car", "take", "monkey", "cash", NULL};

    int total_words = 0;

    // number of total words in words
    for (int i = 0; words[i] != NULL; i++) {
        total_words++;
    }

    probability = (double*) malloc(sizeof(double) *
    (size_t)total_words); // array for probabilities (gets printed
    in the end)
    // size_t converts total_words zu unsigned

    pthread_t thread[total_words]; // array for threads

    int max_threads = 3; ✓
    int active_threads = 0;
    int threads_joined = 0;
```

Fehlende Fehlerbehandlung. -0,5

```

// loop that creates the threads with indexes of the array
and passes them to "thread_activities"
for(int i = 0; i < total_words; i++){

    // arguments for the thread function
    void **args = malloc(sizeof(void*)*2); // array of size 2
for void pointers
    int *index = malloc(sizeof(int));    Siehe oben.

    // pointer to int for index
    *index = i;
    args[0] = index;
    args[1] = words[i];

    // creating one pthread for each loop
    if (pthread_create(&thread[i], NULL, &thread_activities,
args) == 0){
        active_threads++; // if pthread create was successfull ✓
    } else { // if not, wait for the other threads to finish
and then end it
        while(active_threads > 0){
            pthread_join(thread[threads_joined], NULL);
            active_threads--;
            threads_joined++; ✓
        }
        return(EXIT_FAILURE);
    }

    // if 4 threads are running, wait for one to finish bevor
going in the next loop and create another one
    while(active_threads >= max_threads){
        if(pthread_join(thread[threads_joined], NULL) != 0){
            return(EXIT_FAILURE);
        }
        active_threads--; ✓
        threads_joined++;
    }
}

// while there are still threads running, wait for them :
while(active_threads > 0){
    if(pthread_join(thread[threads_joined], NULL) != 0){
        return(EXIT_FAILURE);
    }
    active_threads--; ✓
    threads_joined++;
}

// print the probabilities
for(int i = 0; i < threads_joined; i++){
    printf("%s : %lf\n", words[i],probability[i]); ✓
}

```

```
}  
    // by the way, saying that a yak is less likely to be an  
    animal than a banana, is kinda stupid but i get it  
  
    free(probability);  
    return 0;   
}
```

b & c) 0/3: Keine Bearbeitung.

GBR Übung 04

[PDF](#)

4/4 4.1: Syscall

2/2 a)

C-Bibliothek:

```
#include <sys/time.h>

int main() {
    struct timeval tv;
    for(int i=0; i<100000000; i++) {
        gettimeofday(&tv, NULL); ✓
    }
    return 0;
}
```

Syscall:

```
#include <sys/syscall.h>
#include <unistd.h>
#include <sys/time.h>

int main() {
    struct timeval tv;
    for(int i=0; i<100000000; i++) {
        syscall(SYS_gettimeofday, &tv, NULL); ✓
    }
    return 0;
}
```

1/1 b)

```
• > time ./libc  
./libc 0,14s user 0,00s system 99% cpu 0,138 total  
• > time ./syscall  
./syscall 0,23s user 0,58s system 99% cpu 0,819 total
```

- Das lib-Programm ist deutlich schneller als die direkte syscall-Variante
- Bei lib fällt keine Systemzeit an 
- Beim direkten syscall dominiert die Systemzeit.

-> gettimeofday führt keinen System Aufruf aus

1/1 c)

lib

```
Breakpoint 1, 0x00007ffff7fc4880 in gettimeofday ()
(gdb) step
Single stepping until exit from function gettimeofday,
which has no line number information.
0x000055555555509a in main ()
(gdb) dis
disable      disassemble  disconnect   display
(gdb) dis
disable      disassemble  disconnect   display
(gdb) disassemble
Dump of assembler code for function main:
    0x0000555555555060 <+0>:      push    %rbp
    0x0000555555555061 <+1>:      mov     %rsp,%rbp
    0x0000555555555064 <+4>:      push    %r12
    0x0000555555555066 <+6>:      push    %rbx
    0x0000555555555067 <+7>:      lea     -0x30(%rbp),%r12
    0x000055555555506b <+11>:     sub     $0x20,%rsp
    0x000055555555506f <+15>:     mov     %fs:0x28,%rbx
    0x0000555555555078 <+24>:     mov     %rbx,-0x18(%rbp)
    0x000055555555507c <+28>:     mov     $0x989680,%ebx
    0x0000555555555081 <+33>:     data16 cs nopw 0x0(%rax,%rax,1)
    0x000055555555508c <+44>:     nopl    0x0(%rax)
    0x0000555555555090 <+48>:     xor     %esi,%esi
    0x0000555555555092 <+50>:     mov     %r12,%rdi
    0x0000555555555095 <+53>:     call    0x555555555040 <gettimeofday@plt>
⇒ 0x000055555555509a <+58>:     sub     $0x1,%ebx
    0x000055555555509d <+61>:     jne     0x555555555090 <main+48>
    0x000055555555509f <+63>:     mov     -0x18(%rbp),%rax
    0x00005555555550a3 <+67>:     sub     %fs:0x28,%rax
    0x00005555555550ac <+76>:     jne     0x5555555550b9 <main+89>
    0x00005555555550ae <+78>:     add     $0x20,%rsp
    0x00005555555550b2 <+82>:     xor     %eax,%eax
    0x00005555555550b4 <+84>:     pop     %rbx
    0x00005555555550b5 <+85>:     pop     %r12
    0x00005555555550b7 <+87>:     pop     %rbp
    0x00005555555550b8 <+88>:     ret
    0x00005555555550b9 <+89>:     call    0x555555555030 <__stack_chk_fail@plt>
End of assembler dump.
```



syscall

```
Breakpoint 1 at 0x1064
(gdb) run
Starting program: /home/joshua/Documents/Studium/25-26_WiSe/Betriebssysteme-und-F
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/nix/store/xx7cm72qy2c0643cm1ipngd87aqwkcdp-gli

Breakpoint 1, 0x0000555555555064 in main ()
(gdb) disassemble
Dump of assembler code for function main:
   0x0000555555555060 <+0>:      push    %rbp
   0x0000555555555061 <+1>:      mov     %rsp,%rbp
⇒ 0x0000555555555064 <+4>:      push    %r12
   0x0000555555555066 <+6>:      push    %rbx
   0x0000555555555067 <+7>:      lea     -0x30(%rbp),%r12
   0x000055555555506b <+11>:     sub     $0x20,%rsp
   0x000055555555506f <+15>:     mov     %fs:0x28,%rbx
   0x0000555555555078 <+24>:     mov     %rbx,-0x18(%rbp)
   0x000055555555507c <+28>:     mov     $0x989680,%ebx
   0x0000555555555081 <+33>:     data16 cs nopw 0x0(%rax,%rax,1)
   0x000055555555508c <+44>:     nopl    0x0(%rax)
   0x0000555555555090 <+48>:     xor     %edx,%edx
   0x0000555555555092 <+50>:     xor     %eax,%eax
   0x0000555555555094 <+52>:     mov     %r12,%rsi
   0x0000555555555097 <+55>:     mov     $0x60,%edi
   0x000055555555509c <+60>:     call    0x555555555040 <syscall@plt>
   0x00005555555550a1 <+65>:     sub     $0x1,%ebx
   0x00005555555550a4 <+68>:     jne     0x555555555090 <main+48>
   0x00005555555550a6 <+70>:     mov     -0x18(%rbp),%rax
   0x00005555555550aa <+74>:     sub     %fs:0x28,%rax
   0x00005555555550b3 <+83>:     jne     0x5555555550c0 <main+96>
   0x00005555555550b5 <+85>:     add     $0x20,%rsp
   0x00005555555550b9 <+89>:     xor     %eax,%eax
   0x00005555555550bb <+91>:     pop     %rbx
   0x00005555555550bc <+92>:     pop     %r12
   0x00005555555550be <+94>:     pop     %rbp
   0x00005555555550bf <+95>:     ret
   0x00005555555550c0 <+96>:     call    0x555555555030 <__stack_chk_fail@plt>
End of assembler dump.
```



Der Unterschied wird sichtbar bei libc +53 und syscall +60.

4.2:

a)

- Ausgabe 15:
 - `t1_main` ruft `t1_f1` auf. `x=3` wird auf den Stack gelegt (Offset `-0x4(%rbp)`). `p` wird auf die Adresse von `x` gesetzt. `t1_f1` kehrt zurück.

- `t1_main` ruft `t1_f2` auf. Der Stack-Frame wird an derselben Stelle wie für `f1` erstellt. `y=5` wird an dieselbe relative Adresse gelegt (`-0x4(%rbp)`).
- Gleichzeitig läuft `t2_main . sleep(1)`, was T1 Zeit gibt, `p` zu setzen und `k` zu erhöhen (`k` wird 3).
- T2 dereferenziert `*p`. Da `p` auf die Adresse `-0x4(%rbp)` zeigt (wo früher `x` war und jetzt `y` ist), liest es den Wert 5.
- $k = k * (*p) \rightarrow 3 * 5 = 15$.
- 0, Zufallswerte oder Segfault:
 - Wenn T2 liest, nachdem `t1_f2` bereits zurückgekehrt ist, zeigt `p` in einen "ungültigen" Stackbereich unterhalb des aktuellen Stackpointers (oder in den Frame von `t1_main`). Der Wert ist dann Garbage.
 - Wenn T2 extrem schnell wäre (vor T1s `p=&x`), wäre `p` nicht initialisiert (Global `int* p` ist im BSS 0-initialisiert). Dereferenzierung von `NULL` führt zum Absturz (Segmentation Fault).

b)

Das Verhalten ist undefiniert, da ein "Dangling Pointer" entsteht, weil die globale Variable `p` mit der Adresse der lokalen Variable `x` belegt wird.

- Sobald `t1_f1` endet (return), wird der Speicherbereich von `x` auf dem Stack freigegeben.
- Der Zugriff auf diesen Speicherbereich durch einen anderen Thread (`t2_main`) ist ein klassischer "Use-after-free"-Fehler bzw. Zugriff auf verwaisten Speicher.

4.3 1,5/4

a) 1/2

Zeit	Prozess	Aktion / Codezeile	Var-Änderung	Status
ZS 1	R1	1. DOWN(mutex_str)	mutex_str = 0	OK
		2. DOWN(sem_r)	sem_r = 0	OK
		3. DOWN(mutex_rc)	mutex_rc = 0	OK
ZS 2**	R1	4. readcount++	readcount = 1	
		5. if (readcount == 1) -> DOWN(sem_w)	sem_w = 0	Writer blockiert
		6. UP(mutex_rc)	mutex_rc = 1	
Interrupt		ankunft R2 und W1		
ZS 3	R2	1. DOWN(mutex_str)		Blockiert
ZS 4	W1	1. DOWN(mutex_wc)	mutex_wc = 0	OK
		2. writecount++	writecount = 1	
		3. if (writecount == 1) -> DOWN(sem_r)		Blockiert

Wie sieht der weitere Ablauf aus, bis alle Prozesse fertig sind? -1

=> R1 hält mutex_str und sem_r. R2 wartet auf mutex_str. W1 wartet auf sem_r.

b) 0,5/2

Zeit	Prozess	Aktion / Codezeile	Var-Änderung	Status
ZS 1	W1	1. DOWN(mutex_wc)	mutex_wc = 0	OK
		2. writecount++	writecount = 1	
		3. if (writecount == 1) -> DOWN(sem_r)	sem_r = 0	Lesen gesperrt
Interrupt		ankunft R1		
ZS 2	R1	1. DOWN(mutex_str)	mutex_str = 0	OK
		2. DOWN(sem_r)		Blockiert

W2 trifft während der zweiten Zeitscheibe ein und wird dadurch hinter W1 eingeordnet, sodass W1 vorher rechnen darf. -0,5

Zeit	Prozess	Aktion / Codezeile	Var-Änderung	Status
<i>Interrupt</i>		<i>ankunft W2</i>		
ZS 3	W2	1. DOWN(mutex_wc)		Blockiert

Analog wie bei a): Angabe der Schritte bis alle Prozesse fertig sind. -1

3/3 4.5

2/2 a)

```

) ls -la
dr-x----- joshua users  4 B Sat Dec 13 17:51:45 2025 .
dr-xr-xr-x joshua users  0 B Sat Dec 13 17:51:38 2025 ..
lr-x----- joshua users 64 B Sat Dec 13 17:51:47 2025 0 => /dev/null
lrwx----- joshua users 64 B Sat Dec 13 17:51:47 2025 1 => /dev/pts/0
l-wx----- joshua users 64 B Sat Dec 13 17:51:47 2025 2 => /tmp/err_uid
lrwx----- joshua users 64 B Sat Dec 13 17:51:47 2025 3 => /tmp/tempfileDTuKNe
) ls -la
dr-x----- joshua users  3 B Sat Dec 13 17:51:45 2025 .
dr-xr-xr-x joshua users  0 B Sat Dec 13 17:51:38 2025 ..
lr-x----- joshua users 64 B Sat Dec 13 17:51:47 2025 0 => /dev/null
lrwx----- joshua users 64 B Sat Dec 13 17:51:47 2025 1 => /dev/pts/0
l-wx----- joshua users 64 B Sat Dec 13 17:51:47 2025 2 => /tmp/err_uid

```



Unter `/proc/PID/fd/` befindet:

- 0: Verweist auf `/dev/null` – Standard Input.
- 1: Verweist auf Terminal (`/dev/pts/X`) – Standard Output.
- 2: Verweist auf `/tmp/err_uid` – Standard Error. ✓
- 3: Verweist auf die temp Datei `/tmp/tempfileXXXXXX` .
 - `mkstemp(str3)` erstellt einen Dateideskriptor. Da 0, 1 und 2 belegt sind, ist 3 der nächste freie.

1/1 b)

```
[nix-shell:/proc/52215/task]$ tree */fd
52215/fd
├── 0 → /dev/null
├── 1 → /dev/pts/0
├── 2 → /tmp/err_uid
└── 3 → /tmp/tempfileG48UVX
52216/fd
├── 0 → /dev/null
├── 1 → /dev/pts/0
├── 2 → /tmp/err_uid
└── 3 → /tmp/tempfileG48UVX

2 directories, 8 files

[nix-shell:/proc/52215/task]$ tree */fd
52215/fd
├── 0 → /dev/null
├── 1 → /dev/pts/0
└── 2 → /tmp/err_uid
52216/fd
├── 0 → /dev/null
├── 1 → /dev/pts/0
└── 2 → /tmp/err_uid

2 directories, 6 files
```

- Unterscheiden sich diese während der Ausführung des Programms?
 - Nein. Threads, die mit `pthread_create` erstellt werden, teilen sich (via `CLONE_FILES` Flag beim `clone`-Syscall)  dieselbe Dateideskriptortabelle.
 - Wenn man `/proc/PID/task/TID/fd` für den Hauptthread und den Worker-Thread ansieht, sind die Einträge identisch. 
- Fork statt Thread:
 - Wenn statt `pthread_create` ein `fork()` verwendet wird, würde der Kindprozess eine Kopie der Dateideskriptortabelle  erhalten.

- Wenn der Elternprozess dann `close(fp_ptr)` aufruft, wird nur sein Eintrag entfernt. ✓
- Der Kindprozess hätte den Dateideskriptor weiterhin offen und könnte erfolgreich schreiben. ✓
- => Prozesse haben getrennte Adressräume und Ressourcentabellen (Copy-on-Write bzw. Duplizierung beim Start), Threads teilen sich diese Ressourcen. ✓

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <fcntl.h>
4  #include <sys/mman.h>
5  #include <unistd.h>
6  #include <semaphore.h>
7  #include <time.h>
8  #include <errno.h>
9
10 #define SHM_NAME "/arche_noah_shm"
11 #define SEM_MUTEX "/arche_noah_mutex"
12 #define SEM_FULL "/arche_noah_full"
13 #define N 10
14
15 struct shared_data
16 {
17     int bookings[N];
18     int count;
19 };
20
21 void sem_wait_intr(sem_t *sem)
22 {
23     while (sem_wait(sem) == -1)
24     {
25         if (errno != EINTR)
26         {
27             perror("sem_wait");
28             exit(1);
29         }
30     }
31 }
32
33 int already_used(struct shared_data *data, int id)
34 {
35     for (int i = 0; i < N; i++)
36     {
37         if (data->bookings[i] == id)
38             return 1;
39     }
40     return 0;
41 }
42
43 int main()
44 {
45     srand(time(NULL) ^ getpid());
46
47     int shm_fd = shm_open(SHM_NAME, O_RDWR | O_CREAT, 0666);
48     ftruncate(shm_fd, sizeof(struct shared_data));
49
50     struct shared_data *data = mmap(NULL, sizeof(*data),
51                                     PROT_READ | PROT_WRITE,
52                                     MAP_SHARED, shm_fd, 0);
53
54     sem_t *mutex = sem_open(SEM_MUTEX, O_CREAT, 0666, 1);
55     sem_t *full = sem_open(SEM_FULL, O_CREAT, 0666, 0);
56
57     if (mutex == SEM_FAILED || full == SEM_FAILED)
58     {
59         perror("sem_open");
60         exit(1);
61     }
62
63     sem_wait_intr(mutex);
64
65     if (data->count >= N)
66     {
67         sem_post(mutex);

```

alles schliessen nicht vergessen

```
68         return 0;
69     }
70
71     int id;
72     do
73     {
74         id = rand() % 10000 + 1;
75     } while (already_used(data, id));
76
77     data->bookings[data->count] = id;
78     data->count++;
79
80     printf("Client %d reserves Space %d\n", id, data->count);
81
82     if (data->count == N)
83     {
84         sem_post(full);
85     }
86
87     sem_post(mutex);
88
89     munmap(data, sizeof(*data));
90     close(shm_fd);
91     auch hier alles schliessen
92     return 0;
93 }
94
```



```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <fcntl.h>
4  #include <sys/mman.h>
5  #include <unistd.h>
6  #include <semaphore.h>
7  #include <errno.h>
8
9  #define SHM_NAME "/arche_noah_shm"
10 #define SEM_MUTEX "/arche_noah_mutex"
11 #define SEM_FULL "/arche_noah_full"
12 #define N 10
13
14 struct shared_data
15 {
16     int bookings[N];
17     int count;
18 };
19
20 void sem_wait_intr(sem_t *sem)
21 {
22     while (sem_wait(sem) == -1)
23     {
24         if (errno != EINTR)
25         {
26             perror("sem_wait");
27             exit(1);
28         }
29     }
30 }
31
32 int main()
33 {
34     int shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
35     if (shm_fd == -1)
36     {
37         perror("shm_open");
38         exit(1);
39     }
40
41     ftruncate(shm_fd, sizeof(struct shared_data));
42
43     struct shared_data *data = mmap(NULL, sizeof(*data),
44                                     PROT_READ | PROT_WRITE,
45                                     MAP_SHARED, shm_fd, 0);
46
47     sem_t *mutex = sem_open(SEM_MUTEX, O_CREAT, 0666, 1);
48     sem_t *full = sem_open(SEM_FULL, O_CREAT, 0666, 0);
49
50     if (mutex == SEM_FAILED || full == SEM_FAILED)
51     {
52         perror("sem_open");
53         exit(1);
54     }
55
56     printf("Master waites for complete Buchungsliste...\n");
57
58     sem_wait_intr(full);
59
60     sem_wait_intr(mutex);
61     printf("Buchungsliste complete:\n");
62     for (int i = 0; i < N; i++)
63     {
64         printf("Space %d: Client %d\n", i, data->bookings[i]);
65     }
66     sem_post(mutex);
67

```

alles schliessen nicht vergessen, ansonsten signal richtig behandelt

alles schliessen nicht vergessen (-1)

```
68 | munmap(data, sizeof(*data));
69 | close(shm_fd);
70 |
71 | shm_unlink(SHM_NAME);
72 | sem_unlink(SEM_MUTEX);
73 | sem_unlink(SEM_FULL);
74 |
75 | return 0;
76 | }
77 |
```

list.c

```
/*
 * Simple double linked list implementation
 * created only for educational purposes
 *
 * Task: Operations should be extended with pthread_rwlocklocks
 */
#include "list.h"
#include <stdlib.h>

/*
 * function: dllist_create
 * parameter: dllist ** list: Pointer to the Pointer that should hold the address of the list
 * returns: integer, 0 success, -1 malloc failure
 * description: allocates and initializes the double linked list structure
 */
int dllist_create(dllist** list)
{
    *list = calloc(1, sizeof(dllist));
    if (*list == NULL) {
        return -1;
    }

    // Initialize RW lock
    if (pthread_rwlock_init(&((*list)->lock), NULL) != 0) {
        free(*list);
        return -1;
    }
    return 0;
}

/*
 * function: dllist_free
 * parameter: dllist * list: pointer to the list
 * returns: void
 * description: cleans up all structures
 */
void dllist_free( dllist* list)
{
    // There is no spoon
    if (list==NULL) {
        return;
    }
    while(!dllist_remove(list,0)) {}
}
```

```

    // Destroy RW lock
    pthread_rwlock_destroy(&list->lock);
    free(list);
}

/*
 * function: dllist_insert
 * parameter: dllist * list: pointer to the list, int value: value to be added, int position
 * returns: int: 0 on success, -1 on position out of range, -2 malloc failure, -3 on other error
 * description: inserts an value at position to the list
 */
int dllist_insert(dllist* list, int value, int position)
{
    if ( list == NULL || position < 0 ) {
        return -3;
    }

    // lock for writing
    pthread_rwlock_wrlock(&list->lock);

    dllist_element* list_iterator= list->first; // list element pointer
    //iterate to the right position
    while(position > 1 && list_iterator !=NULL) {
        list_iterator=list_iterator->next;
        position--;
    }

    if (position>1 || (position==1 && list_iterator==NULL)){
        // unlock and return error
        pthread_rwlock_unlock(&list->lock);
        return -1;
    }

    //allocate new element
    dllist_element* new_element = calloc(1,sizeof(dllist_element));
    if(new_element==NULL) {
        pthread_rwlock_unlock(&list->lock);
        return -2;
    }
    new_element->value=value;

    //are we first element
    if(position==0) {
        new_element->next=list->first;
        list->first=new_element;
    } else {

```

Fehlerbehandlung fehlt -0.5
auch bei jedem weiteren lock aufruf

```

        new_element->next=list_iterator->next;
        new_element->prev=list_iterator;
        list_iterator->next=new_element;
    }

    // do we have a next element
    if(new_element->next!=NULL) {
        new_element->next->prev=new_element;
    } else {
        // are we last element
        list->last=new_element;
    }

    // unlock and return success
    pthread_rwlock_unlock(&list->lock);
    return 0;
}

/*
 * function: dllist_get
 * parameter: dllist * list: pointer to the list, int position: list position the value is
 * returns: int: 0 on success, -1 on position out of range, -2 on other errors
 * description: gets an element of the list at the position
 */
int dllist_get(dllist* list, int position, int* result)
{
    if ( list == NULL || position < 0 ) {
        return -2;
    }

    // lock for reading
    pthread_rwlock_rdlock(&list->lock);

    dllist_element* list_iterator= list->first; // list element pointer

    while(position > 0 && list_iterator !=NULL) {
        list_iterator=list_iterator->next;
        position--;
    }

    if (position!=0 || list_iterator == NULL){
        // unlock and return error
        pthread_rwlock_unlock(&list->lock);
        return -1;
    }
}

```

```

        *result=list_iterator->value;

        // unlock and return success
        pthread_rwlock_unlock(&list->lock);
        return 0;
    }

    /*
     * function: dllist_remove
     * parameter: dllist * list: pointer to the list, int position: list position the value is at
     * returns: int: 0 on success, -1 on position out of range, -2 on other errors
     * description: removes an list element at the position
     */
    int dllist_remove(dllist* list, int position)
    {
        if ( list == NULL || position < 0 ) {
            return -3;
        }

        // lock for writing
        pthread_rwlock_wrlock(&list->lock);

        dllist_element* list_iterator= list->first; // list element pointer
        //iterate to the right position
        while(position > 0 && list_iterator !=NULL) {
            list_iterator=list_iterator->next;
            position--;
        }

        if (position!=0 || list_iterator == NULL){
            // unlock and return error
            pthread_rwlock_unlock(&list->lock);
            return -1;
        }

        //is there an element before us
        if (list_iterator->prev != NULL) {
            list_iterator->prev->next= list_iterator->next;
        } else {
            list->first=list_iterator->next;
        }
        if(list_iterator->next != NULL) {
            list_iterator->next->prev= list_iterator->prev;
        } else {
            list->last=list_iterator->prev;
        }
    }

```

```

    }

    free(list_iterator);

    // unlock and return success
    pthread_rwlock_unlock(&list->lock);

    return 0;
}

```

3.5/4

test.c

```

/*
 * Simple test application for the linked list application.
 * created only for educational purposes
 *
 * Task: This test application need to modified that the linked list is tested in multiple p
 */

#include "list.h"
#include <stdio.h>

void print_list(dllist *list){

    int v,r;
    int i=0;
    while(!(r=dllist_get(list,i,&v))) {
        printf("%d:",v);
        i++;
    }
    printf("\n");
}

int main()
{

    int toadd[10]= {7,0,3,0,2,1,9,2,8,4};
    int torm[4]= {0,4,1,0};
    int r;

    dllist* list;
    if(dllist_create(&list)) {
        fprintf(stderr,"Couldn't create list!");
    }
}

```

```

for(int i = 0; i < 10; i+=2) {
    if(dllist_insert(list,toadd[i],toadd[i+1])) {
        printf("couldn't add %d at %d\n",toadd[i],toadd[i+1]);
    }
}

print_list(list);

for(int k = 0; k < 4; k+=1) {
    printf("trying to remove element at position: %d \n", torm[k]);
    if((r=dllist_remove(list,torm[k]))) {
        printf("couldn't remove element at %d ret: %d \n",torm[k],r);
    }
    print_list(list);
}

for(int i = 0; i < 3; i+=2) {
    if(dllist_insert(list,toadd[i],toadd[i+1])) {
        printf("couldn't add %d at %d\n",toadd[i],toadd[i+1]);
    }
}

print_list(list);

dllist_free(list);
}

```

test_all.c nicht vorhanden 0/2

Übungsblatt 5

Montag, 5. Januar 2026 20:38

Yunis Drescher, 827130 Joshua Neunert, 822456 Konrad Gose, 828440

Aufgabe 5.2: Synchronisation

- a) Worin besteht der Unterschied zwischen dem Java- Monitor- und dem Hoare-Monitorkonzept?
- b) Die folgende Umsetzung der put- und get-Funktionen des Java-Cubbyhole-Beispiels ist fehlerhaft. Wie kann man den Fehler beheben?

```
public synchronized int get() {
    if (available == false) {
        try {
            wait();
        } catch (InterruptedException e) {
        }
    }
    available = false;
    notify();
    return contents;
}

public synchronized void put(int value) {
    if (available == true) {
        try {
            wait();
        } catch (InterruptedException e) {
        }
    }
    contents = value;
    available = true;
    notify();
}
```

- c) Überdies kann es auch in der korrigierten Lösung zu Livelocks kommen. Geben Sie ein Beispielszenario dafür an! (1 + 1 + 1 Punkte)

a)

Beim Java-Monitor wird durch notify() ein thread wieder bereit gemacht, aber nicht direkt ausgeführt. Der Thread der notify() ausgeführt hat nehält den Monitor solange bis er ihn verlässt. Der geweckte Thread arbeitet also irgendwann später.

Beim Hoare- Monitor wird der Thread der durch signal(x) aufgerufen wird sofort aktiv und der Thread der signal(x) aufruft wird blockiert. Wenn der geweckte Thread den Monitor verlässt, kann der Signal sendende Thread weiter arbeiten. Dadurch entsteht der Unterschied, dass beim Hoare Monitor der geweckte Thread garantiert seine Bedingung erfüllt hat, während beim Java-Monitor die Bedingung sich wieder geändert haben kann. Daher muss beim Java-Monitor immer wieder mit while die Bedingung geprüft werden.

Hoare verschiedene Bedingungsvariablen mit eigener WS,
Java nur eine WS

0.5/1

c)

Da es sich um Java handelt, müssen wir die Verfügbarkeit nicht nur einmal mit "if" prüfen sondern mit einer while-schleife. Also:

Zeile 2: while (available == false) {
Zeile 12: while (available == true) {
Der Rest des codes kann gleich bleiben.

1/1

d)

Gehen wir davon aus das es mehr als zwei Threads gibt. Da nur notify() und nicht notifyAll() genutzt wird, kann es sein dass ein Thread geweckt wird der direkt wieder blockiert wird. Dieser Thread kann dann kein neues notify() ausführen und alles steht still. Beispiel:

P1, P2, P3

P1, P2 wollen put() nutzen, P2 will get() nutzen.

Wenn available = false, dann wartet P3 in get(). P1 ruft nun put() auf und schreibt seinen wert. Danach setzt er available = true und notify() weckt P2. P2 sieht available = true und geht in wait. Nun ist P1 abgeschlossen und P2 und P3 stecken in der wait() schleife.

livelock

1/1

Aufgabe 5.3: Buddy-System

Ein Betriebssystem verwende das Buddy-System für die Speicherverwaltung mit 2^k KB ($k \geq 0$) großen Blöcken. Der Rechner ist mit 2048 KByte RAM ausgestattet, die vollständig im Buddy-System verwaltet werden.

Die Prozesse A, B, C und D fordern Speicher in folgender Reihenfolge an:

- i) A fordert 236 KByte an.
 - ii) B fordert 144 KByte an.
 - iii) C fordert 76 KByte an.
 - iv) D fordert 456 KByte an.
- a) Zeichnen Sie, nach den Schritten ii) , iii) und iv) die Speicherbelegung ausgehend vom komplett freien und nicht in Blöcke eingeteilten Speicher.
 - b) Wieviele Blöcke sind noch übrig? Wie groß ist dann der größte verfügbare Speicherbereich?
 - c) Geben Sie den aktuellen Verschnitt an!

Nacheinander geben dann die Prozesse B, A, D und C in dieser Reihenfolge ihren Speicher frei.

- d) Zeichnen Sie, ausgehend vom Ergebnis von a), für jeden Schritt die Speicherbelegung. Geben Sie nach jeder Speicherfreigabe den größten zur Verfügung stehenden Block an!

a)

Aufgabe 5.3: 4/4

a) 2/2

Aktion:

o)	2048 KByte						
i)	256 KByte	256 KByte	512 KByte		1024 KByte		
	A:236	✓					
ii)	256 KByte	256 KByte	512 KByte		1024 KByte		
	A:236	B:144	✓				
iii)	256 KByte	256 KByte	128 KB	128 KB	256 KByte	1024 KByte	
	A:236	B:144	C:76	✓			
iv)	256 KByte	256 KByte	128 KB	128 KB	256 KByte	512 KByte	512 KByte
	A:236	B:144	C:76			D: 456	✓

0,5/0,5

b)

Es sind noch 3 Blöcke übrig, wovon der größte Block 512 KByte groß ist



0,5/0,5

c)

Der Verschnitt ist der ungenutzte Speicher in jedem Benutzen Speicherblock, also:

$$(256 - 236) + (256 - 144) + (128 - 76) + (512 - 456) = 240$$



1/1 d)

Entfernen:

		256 KByte	256 KByte	128 KB 128 KB	256 KByte	512 KByte	512 KByte
B		A:236	✓	C:76		D: 456	größter freier Speicher
		512 Kbyte	✓	128 KB 128 KB	256 KByte	512 KByte	512 KByte
A		größter freier Speicher		C:76		D: 456	größter freier Speicher
		512 Kbyte		128 KB 128 KB	256 KByte	1024 KByte	
D				C:76		größter freier Speicher	✓
		2048 KByte					
C		größter freier Speicher ✓					



Aufgabe 5.4: Gutartigkeit

Zeigen Sie, daß LRU ein gutartiger Seitenersetzungsalgorithmus ist!

(2 Punkte)

Beweis über Stack-Algorithmus:

Ein Algorithmus ist gutartig, wenn er ein Stack-Algorithmus ist.
 Ein Algorithmus ist ein Stack-Algorithmus, wenn es eine feste Prioritätsordnung gibt mit der die Seiten eingeladen werden.

Also gilt in Bezug auf Gutartigkeit, dass:

- Der Speicher der Größe m enthält immer die m höchstpriorisierten Seiten
- Bei größerem Speicher werden nur "unten" zusätzliche Seiten ergänzt

Da LRU eine feste Prioritätsordnung hat, höchste Priorität der Ersetzung für die am längsten nicht genutzte Seite und somit die Anforderung eines Stack-Algorithmus erfüllt ist er gutartig

Aufgabe 5.5: Speicherverwaltung Seitenfehler 2/2

Die Behandlung eines Seitenfehlers erfordert i. A. Maßnahmen zur Ersetzung einer Seite in einem Seitenrahmen (Kachel) des Arbeitsspeichers. Dazu muss zunächst eine andere Seite aus dem Arbeitsspeicher verdrängt werden.

Bei der Ausführung eines Benutzerprogramms tritt folgende Sequenz von Seitenzugriffen auf:

7 6 2 7 5 5 3 2 7 4 5

Bestimmen Sie die minimale Anzahl der Seitenfehler, die auftreten kann! Gehen Sie dabei von 4 Seitenrahmen aus und begründen Sie Ihre Antwort. (2 Punkte)

Die Minimale Anzahl von Seitenfehler erhält man über den Belady-Algorithmus, da er die Seiten die man als nächstes Braucht kennt und somit immer optimal löscht. Dieser ist aber nur möglich wenn man die Sequenz kennt bevor sie passiert. ✓

SW	7	6	2	7	5	5	3	2	7	4	5
1	7	7	7	7	7	7	7	7	7	4	4
2		6	6	6	6	6	3	3	3	3	3
3			2	2	2	2	2	2	2	2	2
4					5	5	5	5	5	5	5
SF	*	*	*		*		*			*	

Die minimale Anzahl der Seitenfehler ist also 6 ✓

3/3 Aufgabe 5.6: Speicherverwaltung Algorithmen

Es steht ein physikalischer Adressraum mit 4 Seitenrahmen zur Verfügung. Arbeiten Sie die angegebene Seitenzugriffsfolge gemäß der Seitenersetzungsstrategien:

- FIFO (First-In-First-Out),
- LRU (Least-Recently-Used),
- Auslagern einer Seite, die nicht zum Working-Set gehört (Working-Set-Parameter $T=3$).

ab, und vergleichen Sie diese anhand der Seitenfehleranzahl!

2 7 6 7 1 4 7 2 5 5 1

In denjenigen Fällen, in denen Nicht-Determiniertheit vorliegt, ist stets die Seite mit der kleinsten Seitenrahmennummer zu wählen, die das jeweilige Verfahren erfüllt.

(3 Punkte)

FIFO											
SW	2	7	6	7	1	4	7	2	5	5	1
Jung	2	7	6	6	1	4	4	2	5	5	5
		2	7	7	6	1	1	4	2	2	2
			2	2	7	6	6	1	4	4	4
Alt					2	7	7	6	1	1	1
SF	*	*	*		*	*		*	*		



LRU											
SW	2	7	6	7	1	4	7	2	5	5	1
1	2	2	2	2	2	4	4	4	4	4	1
2		7	7	7	7	7	7	7	7	7	7
3			6	6	6	6	6	2	2	2	2
4					1	1	1	1	5	5	5
SF	*	*	*		*	*		*	*		*



T=3											
SW	2	7	6	7	1	4	7	2	5	5	1
1	2	2	2	2	2	4	4	4	4	4	1
2		7	7	7	7	7	7	7	7	7	7
3			6	6	6	6	6	2	2	2	2
4					1	1	1	1	5	5	5
SF	*	*	*		*	*		*	*		*



FIFO hat 7 Seitenfehler und LRU und Working Set haben 8 Seitenfehler.



4/4 Aufgabe 6.1: Fehlererkennung & -korrektur

2/2 a) Die folgende Bitfolge soll über ein Netzwerk übertragen werden:

1011101

Ermitteln Sie die Bitfolge, die unter Benutzung des Hammingcodes übertragen wird!

2/2 b) Sie empfangen die folgende Bitfolge, die den Hammingcode benutzt:

00111010000

Bestimmen Sie, ob ein Übertragungsfehler aufgetreten war. Wenn ja, bestimmen Sie die Bitposition im Original, an der der Fehler auftrat.

Achtung: Die Bitfolgen von a) und b) haben nichts miteinander zu tun.

(2+2 Punkte)

a)

Da $m = 7$, muss $r^2 \geq 7+r+1$ sein

Also: $r^2 \geq 8+r$

Sei $r = 3$: $9 \geq 11$

Sei $r = 4$: $16 \geq 12$

pos	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100
Bit vor	P1	P2	1	P3	0	1	1	P4	1	0	1	
Bit nach	0	0	1	0	0	1	1	0	1	0	1	

Für P1 wird 0011, 0101, 0111, 1001, 1011 geprüft, es gilt:

$$1+0+1+1+1 = 4 \mod 2 = 0 \Rightarrow P1 = 0$$

Für P2 wird 0011, 0110, 0111, 1010, 1011 geprüft, es gilt:

$$1+1+1+0+1 = 4 \mod 2 = 0 \Rightarrow P2 = 0$$

Für P3 wird 01001, 0110, 0111 geprüft, es gilt:

$$0+1+1 = 2 \mod 2 = 0 \Rightarrow P3 = 0$$

Für P4 wird 1001, 1010, 1011 geprüft, es gilt:

$$1+0+1 = 2 \mod 2 = 0 \Rightarrow P4 = 0$$

Also wird folgendes übertragen:

00100110101

b)

pos	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011
Bit nach	0	0	1	1	1	0	1	0	0	0	0
Bit soll	P1 = 0	P2 = 0	1	P3 = 1	1	0	1	P4 = 0	0	0	0
Bit vor											

P1 sagt uns, dass:

0011, 0101, 0111, 1001, 1011 eine grade Anzahl von 1 haben muss, prüfen:
 $1+1+1+0+0 = 3 \mod 2 = 1$, also haben wir an mind. einer der Stellen ein Fehler

P2 sagt uns, dass:

0011, 0110, 0111, 1010, 1011 eine grade Anzahl von 1 haben muss, prüfen:
 $1+0+1+0+0 = 2 \mod 2 = 0$, also haben wir keinen Fehler an diesen Stellen

Das schließt aus P1 aus und wir haben nun noch einen Fehler an diesen Stellen:

0101,1001

P3 sagt uns, dass:

0101, 0110, 0111 eine ungrade Anzahl von 1 haben muss, prüfen:

$1+0+1 = 2 \mod 2 = 0$, also haben wir hier einen Fehler ✓

Aber aus P2 wissen wir, dass bei 0110 und 0111 kein Fehler ist, also haben wir bei 0101 einen Fehler

P4 sagt uns, dass:

1001, 1010, 1011 eine gerade Anzahl von 1 haben muss, prüfen:
 $0+0+0 = 0 \mod 2 = 0$, also haben wir hier auch keine Fehler. ✓

Fügen wir das alles zusammen, erhalten wir, dass von P3 nur bei 0101 ein Fehler sein kann.

Von P1 wissen wir, dass bei

0011, 0101, 0111, 1001, 1011 ein Fehler sein muss, streichen wir Duplikate wo wir wissen dass sie richtig sind, erhalten wir: 0101

Also stimmen P1 und P3 überein und wir haben einen Fehler an der Stelle 0101 = 1

Es ist natürlich auch möglich dass mehrere Fehler aufgetreten sind, aber das lässt sich nicht prüfen

Alternativweg über Syndrom:

Bei P1 ist ein Fehler $\Rightarrow$ Syndrom = 1

Bei P3 ist ein Fehler $\Rightarrow$ Syndrom = $1 + 4$, da P3 an der 4ten Stelle steht

Syndrom = 5, also ist ein Fehler an der 5ten Stelle, oder 0101 ✓

Damit wissen wir, dass die eigentliche Nachricht so aussah:

1001000

Aufgabe 6.2: HDLC und CRC

Im HDLC (High-level Data Link Control) werden die Nutzdaten in sogenannte Frames verpackt. Der Beginn und das Ende eines solchen Frames ist durch je ein Flag bestehend aus der Bitfolge 0111 1110 gekennzeichnet. Wir betrachten nun eine vereinfachte Version eines solchen Frames:

Zwischen den beiden Flags werden die Nutzdaten und das CRC-Feld zur zyklischen Blocksicherung übertragen. Ein HDLC-Frame habe also folgendes Aussehen:

0111 1110	Nutzdaten	CRC	0111 1110
-----------	-----------	-----	-----------

- a) i) Es soll die Bitfolge 1011 1110 1010 1010 als Nutzdaten übertragen werden. Als Generatorpolynom werde $G(x) = x^8 + x^7 + x + 1$ verwendet.

Bestimmen Sie die Bitfolge, die im CRC-Feld übertragen wird, und geben Sie den vollständigen HDLC-Frame an!

- ii) Erklären Sie das Bitstopfen anhand der vorgegebenen Rahmenflags. Nehmen Sie zusätzlich an, innerhalb von zu versendenden Nutzdaten befindet sich eine Bitsequenz der Art {0111 1100 1100}. Begründen Sie, ob Sie das Bitstopfen für diesen Fall durchführen müssen.

- b) Wir gehen jetzt von folgendem Generatorpolynom aus: $G(x) = x^4 + x + 1$. Wurde die Nachricht 1101 0110 1110 korrekt empfangen? Begründen Sie!

(3 + 1 + 2 Punkte)

a)

i)

Da der höchste Exponent des Polynoms 8 ist, nutzen wir CRC-8.
Das bedeutet wir müssen an die Nutzdaten acht 0 anhängen

Also haben wir:

Nutzdaten: 1011 1110 1010 1010 0000 0000

Das Generatorpolynom ist:

1 1000 0011

3,5/4 Aufgabe 6.3: Schiebefensterprotokolle

Es gibt unter dem angegebenen Link einen Simulator für die Schiebefensterprotokolle Go-Back-N und Selektive Repeat: https://www2.tkn.tu-berlin.de/teaching/rn/animations/gbn_sr/.

Starten Sie den Simulator mit folgender Konfiguration:

```
window size = 4
end to end delay = 5000
timeout = 11000
number of packets emitted per minute = 60
```

Starten Sie folgendes Szenario:

- Der Sender schickt die ersten 4 Rahmen los.
- Für Rahmen mit der Nummer 3 tritt eine Störung auf. Er geht während der Übertragung verloren.

Beantworten Sie folgende Fragen für beide Protokolle!

- 1/1 a) Wann werden wieviele ACKs geschickt, um den Fehler zu markieren?
- 1/1 b) Wie reagiert der Sender und wann?
- 0,5/1 c) Falls end to end delay wegen Überlast auf 6000 erhöht werden muss, wie muss timeout eingestellt sein um einen reibungslosen Ablauf zu gewährleisten?
- 1/1 d) Wird der Unterschied zwischen den beiden Protokollen in der Simulation sichtbar? Begründen Sie!

(4 Punkte)

Go-Back-N

a)

Es werden für jeden erhaltenen Rahmen der nicht in der Richtigen Reihenfolge eintrifft ein weiteres Ack von dem letzten Ack was erhalten wurde gesendet, also R2. In diesem Beispiel sendet R2 seinen eigenen Ack und dann 3 Ack's für R4, R5, R6 gesendet 

b)

Nachdem dann der verloren gegangene Rahmen ein Timeout hat, werden alle Rahmen die aktuell im Fenster sind neu gesendet. Danach arbeitet er normal weiter 

c)

Wenn end to end delay auf 6000 gesetzt wird, führt das dazu das beim übertragen des ersten Fensters alle Rahmen zweimal gesendet werden. Damit das nicht passiert, muss timeout mindestens auf 12500

$\text{Timeout} > 12000 = 2 \times \text{End-to-End-Delay}$ -0,5

Selektiv Repeat

a)

Es werden keine besonderen Ack's gesendet. Jeder Rahmen sendet sein eigenes Ack 

b)

Nachdem der Rahmen einen Timeout hat, sendet er neu. Nachdem der verlorene Rahmen erhalten wird, bewegt sich das Fenster weiter. 

c)

Jedes Paket muss zweimal gesendet werden. Um das zu beheben, muss timeout auf mindestens auf 12500 gesetzt werden

Wie bei Go-Back-N.

Die Simulation zeigt, dass Selektiv repeat weniger Daten neu sendet wenn ein fehler auftritt. Es werden aber keine weiteren Unterschiede sichtbar. Erhöht man aber die Window size, sieht man das Selectiv repeat schneller ist, da die nicht verlorenen Daten weiter erhalten werden und durch die Große Fenster Größe ein kleinerer Zeitunterschied zwischen Erneutem senden und Erhalten des verlorenen Rahmen und dem Erhalten des letzten Rahmen des Fensters. Also sieht man keinen Unterschied, da das Window zu klein ist. 

2/2 Aufgabe 6.4: RFCs

Der RFC 2119 definiert die Begriffe must, shall, should, may, etc.. Prüfen Sie, ob der gegebene Pseudo-Code standardkonform (im Sinne des RFC 2119) zum Ausschnitt eines RFCs implementiert ist! Beschreiben Sie kurz den Unterschied zwischen SHALL und SHOULD!

SHALL ist ein anderes Wort für MUST, also ist es eine Anforderung der

Spezifikation die nicht ausgelassen werden kann.

SHOULD bedeutet es kann unter bestimmten Umständen weggelassen werden, aber man sollte es trotzdem enthalten, wenn man sich nicht sicher ist



Auszug aus dem RFC 2324 (Hyper Text Coffee Pot Control Protocol)

2.3.2 418 I'm a teapot

Any attempt to brew coffee with a teapot should result in the error code "418 I'm a teapot". The resulting entity body MAY be short and stout.

Eine Implementierung:

```
BREW(container, coffee)
{
  combine(container, filter);
  combine(filter, coffee);
  do {
    heat(water);
  } while (state(water) != hot);
  result = insert(container, water);
  return result;
}
```

(2 Punkte)

Der Pseudocode kann Standardkonform sein. Da es keinen Fehler "418 I'm a teapot" implementiert, dies aber durch das should unter bestimmten Umständen weggelassen werden kann, kann es sein dass es Standardkonform ist, wenn es in diesem Fall berechtigt weggelassen wird.



Aufgabe 6.7: Rechnernetze: Wegwahl

Gegeben sei die folgende Wegetabelle eines Linux-Routers:

Zielnetz-IP in CIDR-Notation	Gateway	Interface
default/0	192.168.178.254	eth0
192.168.178.0/24 <i>0-255</i>	0.0.0.0	eth0
141.89.64.0/25 <i>0-127</i>	91.66.76.254	eth0
10.100.0.0/12 <i>96.0.0-111.111.255</i>	0.0.0.0	wg0
10.100.4.0/22 <i>4.0-7.255</i>	0.0.0.0	docker0
141.89.64.70/32	10.100.7.2	docker0

- a) Der Linux-Router besitzt 3 Ethernet-Schnittstellen eth0, docker0 und wg0. Es sollen Pakete an die folgenden Ziele verschickt werden:

- 141.89.58.20 *111 110 000 1100*
- 1.1.1.1 *000 0100*
- 141.89.64.70
- 10.100.1.12
- 10.100.4.1
- 141.89.64.112

Geben Sie für jedes Paket an, ob es eine valide Route in der Wegwahltabelle existiert! Falls es eine gibt, geben Sie das Interface an, über welches das Paket verschickt wird und welche MAC-Adresse in dem zugehörigen Ethernetrahmen steht. Sollte eine relevante Route in der Wegwahltabelle nicht valide sein, begründen Sie ihre Entscheidung!

- b) Die Tools ping und traceroute wurden in der Vorlesung vorgestellt. Nennen Sie eine Gemeinsamkeit von ping und traceroute! Beschreiben Sie außerdem wie traceroute den Pfad, den ein Paket nimmt, herausfindet!

(3+1 Punkte)

a)

Default wird über 192.168.178.254 erreicht, welches in 192.168.178.0/24 enthalten ist

Ziel: 141.89.58.20
Nicht in 141.89.64.0/25 enthalten
=> default /0
Default wird über 192.168.178.254 erreicht, welches in 192.168.178.0/24
enthalten ist

Also ist die Route über eth0 zu 192.168.172.254 über eth0 zu default.
Die MAC-Adresse ist die von 192.168.172.254

Ziel 1.1.1.1
Über default/0

Also ist die Route über eth0 zu 192.168.172.254 über eth0 zu default.
Die MAC-Adresse ist die von 192.168.172.254

Ziel: 141.89.64.70
Routen: 141.89.64.0/25
141.89.64.70/32
Wir entscheiden uns für 141.89.64.70/32 da es den längeren Präfix hat
141.89.64.70/32 geht über das Gateway 10.100.7.2 welches in 10.100.4.0/22
enthalten ist

Also ist die Route über docker0 zu 10.100.4.0/22 über docker0 zu 141.89.64.70
Die MAC-Adresse ist die von 10.100.7.2

Ziel 10.100.1.12
Route: 10.100.0.0/12
Das Gateway ist 0.0.0.0
Also ist die Adresse direkt erreichbar
MAC-Adresse ist die von 10.100.1.12

Ziel 10.100.4.1
Route 10.100.4.0/22 da es den längeren Präfix hat
Da es auch das Gateway 0.0.0.0 hat ist es auch direkt erreichbar
MAC-Adresse ist die von 10.100.4.1

Ziel: 141.89.64.112
Route: 141.89.64.0/25
Gateway: 91.66.76.254 **daher über default**
Das Gateway ist nicht Erreichbar in eth0, da es nicht in 192.168.178.0/24 liegt.
Da das Gateway nicht Erreichbar ist, ist die Route nicht erfüllbar

2.5/3

b)
Beide Tools verwenden ICMP-Nachrichten um die IP-Netze zu erfragen

traceroute funktioniert dadurch, dass ein Router ein Paket verwirft wenn der TTL
zähler 0 erreicht. Wenn er es verwirft sendet es an den Absender eine ICMP Time
Exceed nachricht mit der IP-Adresse des Routers.
traceroute funktioniert indem es ein Paket mit TTL= 1 sendet. Der Erste Router
reduziert TTL auf 0 und verwirft das Paket sendet nun ein ICMP Time Exceed.
traceroute kennt nun den ersten Router. Es wiederholt das nun immer mit um 1
inkrementiertem TTL. Sobald das Ziel erreicht ist läuft TTL nichtmehr auf 0 und
antwortet stattdessen mit einem ICMP Echo Reply. Dadurch weist traceroute das
es das Ziel erreicht hat und alle Router auf dem Weg zum Ziel kennt. Die erhaltene
Reihenfolge ist auch die Reihenfolge des tatsächlichen Wegs

1/1

GBR Übung 06

6

6 2/4

a) 0/1

Einige Protokolle fehlen: Ethernet, UDP, IPv4.
1 Punkt nur bei Nennung aller relevanten Protokolle.

Protokoll	OSI-Schicht	Name der Schicht
HTTP	Schicht 7	Anwendung (Application)
DNS	Schicht 7	Anwendung (Application)
TCP	Schicht 4	Transport (Transport)
STP	Schicht 2	Sicherung (Data Link)
LLDP	Schicht 2	Sicherung (Data Link)

b) 1/1

Ethernet

Source (Address A)	Destination (Address B)	Packets	Bytes
9c:05:91:70:5d:dc	01:80:c2:00:00:00	8	512
9c:05:91:70:5d:dc	01:80:c2:00:00:0e	1	169
a0:88:c2:b5:f4:5a	a0:88:c2:b6:14:1a	2,449	3,563,619



IP(v4)

Host A	Host B	Packets	Total Bytes
10.3.10.45	10.3.10.43	2,449	3,563,619



TCP

Source (IP:Port)	Destination (IP:Port)	Packets	Bytes
10.3.10.45:43044	10.3.10.43:80	2,431	3,561,309
10.3.10.45:43058	10.3.10.43:80	10	1,638



UDP

Source (IP:Port)	Destination (IP:Port)	Packets	Bytes
10.3.10.45:38025	10.3.10.43:53	4	336
10.3.10.45:58386	10.3.10.43:53	4	336



c) 0/1

```

0,348790
+ 0,000012
+ 0,815977
+ 0,000024
+ 0,000139
+ 0,000054
+ 0,000113
+ 0,000041
-----

```

1,16515 / 8 ~= 0,14564375s

Durchschnittliche Latenzzeit für
DNS-Anfragen: 0,16 ms -0,5

In Wireshark: Statistics -> DNS
-> Query Responses

http: 0.000029s

Latenz des HTTP-Requests: 0,179588 s
Pakete 14 und 2436 betrachten. -0,5

d) 1/1